

Network Administration / System Administration

Homework 12

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

Discussion Partners:

- Classmates：賴泓安、鄭竣陽、喻慈恩

1 Linux 大小事

Reference:

- Linux 的 `/etc/shadow` 檔案結構：儲存真實密碼的地方
- <https://hackmd.io/@ncnu-opensource/ry8-hpefi>
- <https://www.cyberciti.biz/faq/understanding-etcshadow-file/>
- <https://dywang.csie.cyut.edu.tw/dywang/linuxsecurity/node39.html>
- https://linux.vbird.org/linux_basic_train/centos7/unit05.php#5.3
- How To Monitor System Authentication Logs on Ubuntu
- <https://www.tecmint.com/find-failed-ssh-login-attempts-in-linux/>
- 使用 fail2ban 配置保護您的 Linux 伺服器

1. (a) Linux 密碼儲存方式

在目前 Linux 系統當中，跟密碼有關的有兩個檔案：`/etc/passwd` 和 `/etc/shadow`。這兩個檔案的差異在於：

- `/etc/passwd`：儲存使用者帳號、UID、GID、使用者全名、家目錄和 shell 等資訊，但密碼欄位通常只包含一個特殊字元（如 `x` 或 `*`），表示密碼已被儲存在 `/etc/shadow` 中。
- `/etc/shadow`：專門用來儲存使用者的密碼資訊，包括加密後的密碼、密碼過期時間等。這個檔案只有 root 使用者可以讀取。

而在 `/etc/shadow` 中，每一行由多格經由冒號分隔的欄位組成。在第二欄當中，存的是加密後的密碼。這個欄位的格式通常是 `$<hash type>$<salt>$<hashed password>`。

1. (b) passwd 原理

passwd 這個指令有一個特殊的權限叫做 SUID (Set User ID)。透過查看 `/etc/bin/passwd` 的權限，可以發現它的權限是 `rwsr-xr-x`，其中的 `s` 表示這個檔案具有 SUID 權限。並且，這個檔案是被 `root` 所擁有。當使用者執行這個指令時，因為 SUID 的緣故，系統會暫時將使用者的權限提升到該檔案擁有者的權限，也就是 `root`，因此這樣就可以修改 `/etc/shadow` 檔案中的密碼。

1. (c) 登入 log

登入的 log 記錄通常位於 `/var/log/auth.log` 檔案當中。這個 log 檔案會記載所有嘗試的登入事件，包含 `sudo`、`su`、SSH 登入等。每一行通常包含時間戳、動作、使用者名稱、來源 IP 等資訊。

1. (d) 防範密碼破解

- 第一個防範方式是使用 `fail2ban` 這類軟體。`fail2ban` 會監控登入失敗的事件，並且在檢測到可能是惡意登錄的情況下，透過修改防火牆規則來封鎖該 IP 地址。這樣可以有效地防止暴力破解攻擊。
- 第二個防範方式較為簡單暴力，就是乾脆不要用密碼來登入，只允許使用 SSH 金鑰的方式來登入就好。這種方法的設置方式是透過在 `/etc/ssh/sshd_config` 中，將 `PasswordAuthentication` 設為 `no`。這樣就會禁止使用密碼登入，只允許使用 SSH 金鑰登入。

2 畫中有話

Reference:

- <https://ctf101.org/forensics/what-is-steganography/>

2. (a) hide.py 運作原理

`hide.py` 是以 LSB Steganography 的方式將訊息隱藏在圖片中，方法如下：

1. 首先，程式使用 Python 的 PIL 套件來將圖片每一個像素的 RGB 值讀取出來，並且存成一個陣列。
2. 將訊息轉換成 8 位元的二進位制格式。
3. 每一個字元的訊息會經由三個像素點的 RGB 值，總共 9 個位元來儲存。他的儲存方式是將每個像素的 RGB 值的最後一個位元 (LSB) 改成訊息的二進位制位元。
4. 最後再將圖片的像素值寫回到圖片中，並且儲存成新的圖片檔案。

2. (b) 圖中的訊息

這題的破解方式非常簡單，就按照 `hide.py` 程式碼的邏輯反向操作就好。我撰寫了 `seek.py` 這個程式來讀取圖片中的像素值，並且將每個像素的 RGB 值的最後一個位元 (LSB) 組合起來，然後轉換成字元。最後就可以得到原本被隱藏在圖片中的訊息。

Flag:

HW12{S4KiCh4n_sakiCHAN_S4k1ChaN}

3 Alya Judge

Reference:

- <https://www.cnblogs.com/znh8/p/9316923.html>
- <https://www.begtut.com/python/ref-requests-post.html>
- <https://www.maxlist.xyz/2019/06/29/flask-session/>
- <https://ithelp.ithome.com.tw/articles/10318320?sc=rss.iron>

3. (a) fysty 的上傳紀錄

Steps:

1. 根據題目，我們可以知道必須要使用 `path traversal` 攻擊來想辦法找到帳號相關資訊。
2. 在題目給的 `app.py` 當中，可以發現帳號資訊應該是存在 `accounts/accounts.json` 這個檔案當中。
3. 我請 AI 幫我分析 `app.py` 的程式碼，幫我找到哪一個網址的請求會被直接解析，來讓我有漏洞可以使用 `path traversal` 攻擊。在分析後，發現 `submissions/<username>` 開頭的路徑會被轉去以 `load_submissions` 函式直接獲得從 `submissions/` 開始，相對路徑為 `<username>` 的資料，但 `load_submissions` 函數會直接用 `load_data` 函式來開啟該路徑的檔案，不會做任何的檢查。
4. 因此，我就要想辦法在網址列裡面輸入 `submissions/../accounts/accounts.json` 來達到 `path traversal` 的效果。這樣就可以讀取到 `accounts.json` 檔案的內容。
5. 我嘗試了很多次，一直得不到正確的結果，後來發現是因為 `load_submissions` 函式在處理路徑時，會自動補上 `json` 副檔名，所以我只要輸入前面的內容就好。
6. 最後，我成功用以下的網址來讀取 `accounts.json` 的內容：

```
1 http://140.112.91.4:45510/submissions/%2e%2e%2faccounts%2faccounts
```

7. 根據獲得到的帳密，我可以得到 `fysty` 的密碼 `hash` 值為
`40c3d69c8a012e181bd63d215d61a1df44e8fe7c182da6d24f26b0fae5348010`。

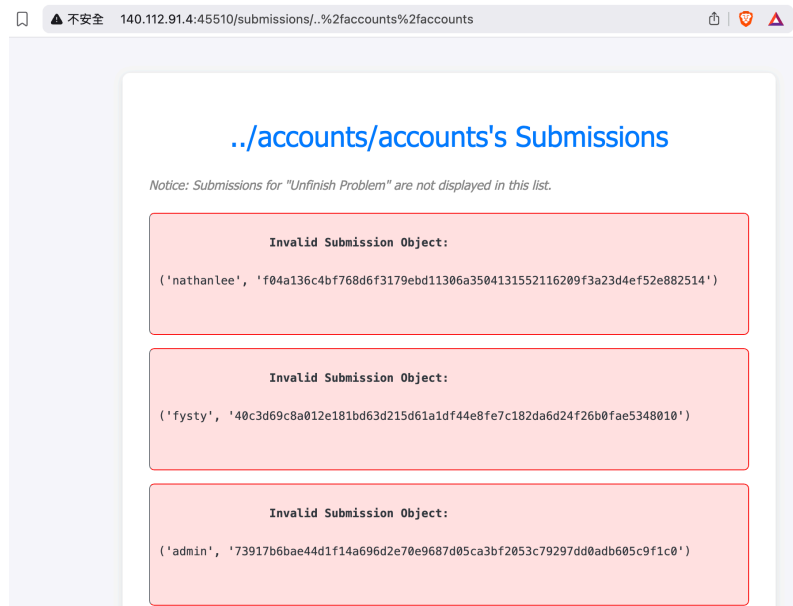


Figure 1: 成功讀取到帳號資訊

8. 我發現在 `app.py` 當中，密碼的 hash 方式為以下：

```
1 hashlib.sha256(password.encode("utf-8")).hexdigest():
```

因此，我撰寫了 `brute-force-passwd.py` 來從給予的密碼清單當中暴力破解密碼。

9. 我找到 `fysty` 的密碼為 `mortis00`，並且成功登入。

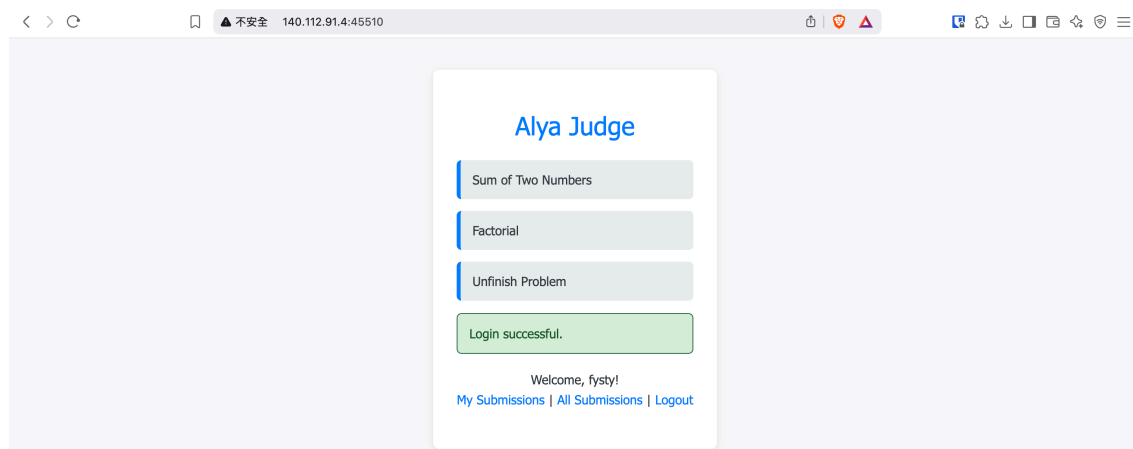


Figure 2: 成功登入 fysty 的帳號

10. 登入之後，我可以看到 `fysty` 的上傳紀錄當中包含了 Flag。如下所示：

Flag:
 HW12{r3MeM8eR_To_s3t_S7R0Ng_PAS5W0rds}

3. (b) 未完成的題目公開了

Steps:

1. 觀察 `app.py` 以及多上傳幾次之後，我發現 Special Judge 的運作原理是直接將你的 `submission` 程式跟 `solution` 陣列當中的每一個字串做比對，而且當你傳字元越正確時，會回傳的分數就越高。
2. 因此，我可以透過 `post-submit.py` 這個程式來自動化地嘗試每一個字元，直到找到正確的 Flag。這個程式會從 `HW12{` 開始，然後逐一嘗試每一個字元，直到找到正確的 Flag 為止。
3. 然而，我發現我不知道我要怎麼 POST，所以我用瀏覽器的開發者工具來觀察網路請求，發現當我上傳程式碼時，會發出一個 POST 請求到 `/submit/3` 路徑。這個請求的內容包含為 `language=python&code=HW12` 這樣的內容。我發現我可以將我要查的東西弄成一個 json 格式，然後用 `resp = requests.post(URL, data=payload)` 來發送 POST 請求。
4. POST 完之後，他會跳轉到一個 HTML 頁面，這個頁面會包含一個 flash 訊息，裡面會有我上傳的程式碼的結果。這個 flash 訊息的格式是 `Submission Result for Problem 3: result, score`。我請 GPT 幫我撰寫了一個正規表達式來抓取這個訊息，並且從中提取出結果和分數。
5. 最後，我得到了正確的 Flag，並成功提交得到 AC。

Flag:

HW12{e5x5Vw2qC}

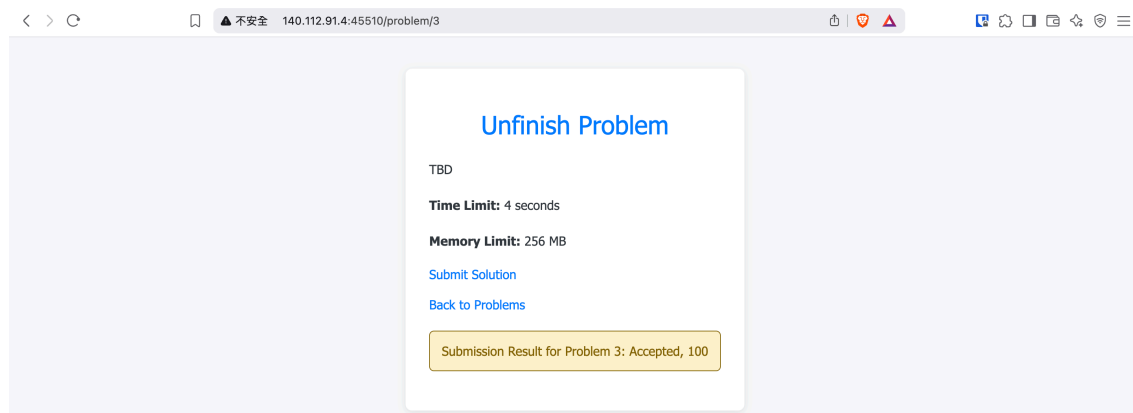


Figure 3: 成功提交得到 AC

3. (c) admin 的上傳紀錄

Steps:

1. 由於題目有提示要使用 Cookie 來想辦法取得 admin 權限，因此我研究了 flask 是怎麼實作 session 的。我發現 flask 的 session 是存在 client 端的 Cookie 中，並且透過 `app.secret_key` 來加密的。

- 我發現 secret key 是在 app.py 中明碼定義的。由於這個 key 是公開的，因此我可以直接使用這個 key 來加/解密 Cookie 中的 session 資料。但是，我不會寫 session 的加解密，因此我想到我可以將整個 OJ 在我本地端跑起來，然後修改 accounts.json 檔案，將 admin 的密碼改成跟第一小題 fysty 一樣的密碼 hash 值，然後再以 admin/mortis00 登入一次。這樣就可以在我的本地端拿到 admin 的 session cookie 資料。
- 我將 flask app 跑起來之後，連上 `http://127.0.0.1:45588/`，進到瀏覽器開發工具的 Application 分頁，找到 Cookies 的部分，然後將 session 的值複製下來。這個值為：

`eyJ1c2VybmFtZSI6ImFkbWluIn0.aEWGTW.6DVuV2HJdC0iTJC5wgxCzMV0xVw`
- 接著，我重新連回 Judge 上，先以 fysty 登入，然後進到開發者工具，將這個 Cookie 的值貼上去，然後刷新頁面。這樣就可以以 admin 的身份登入了。

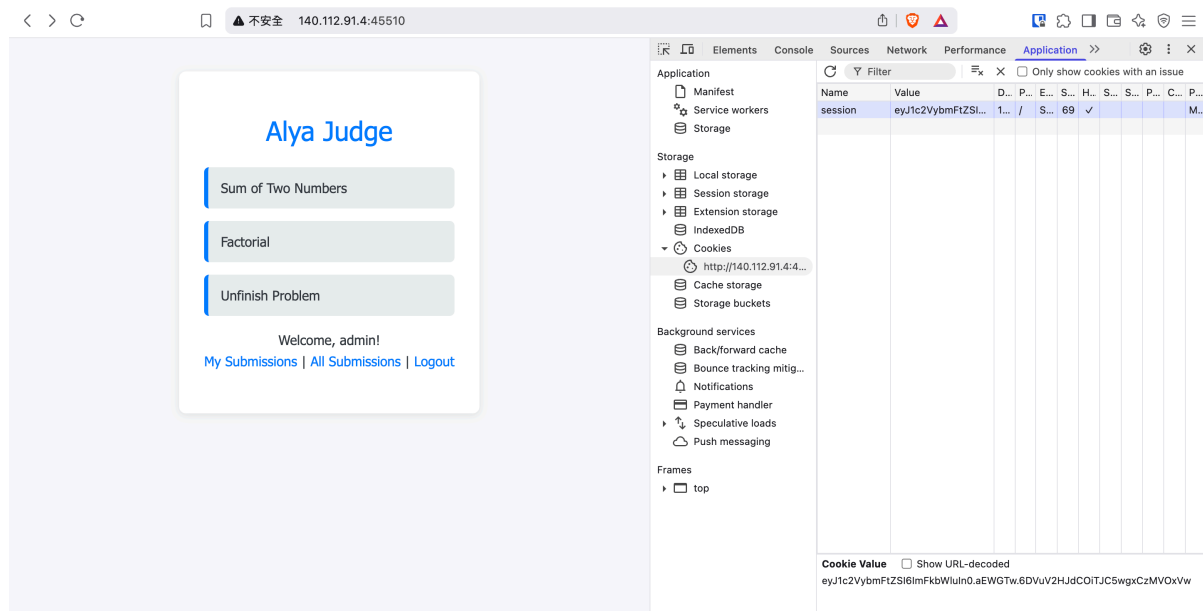


Figure 4: 成功以 admin 身份登入

- 進到 My Submissions 頁面後，我成功在 Unfinish Problems 的上傳紀錄中看到 Flag。如下所示：

Flag:

`HW12{i_l1KE_a15CR3am_MoRE_7H4n_Co0Ki3s}`

```

Problem: Unfinish Problem
Language: python
Result: Wrong Answer
Score: -1
Code:
HW12{i_l1KE_a15CR3am_MoRE_7H4n_Co0Ki3s}

```

Figure 5: 成功在 admin 的上傳紀錄中看到 Flag

4 Introduction to gnireenignE esreveR

Reference:

- <https://dogbolt.org/faq>
- <https://wangchuijiang.com/linux-command/c/hexdump.html>

Steps:

1. 我嘗試使用 dogbolt 這個網站來反編譯 chal.exe。在他 Reverse 出來的眾多檔案當中，我發現 Hex-Rays 解出來的 C 語言程式碼是最完整的，可以看到 main 函數的邏輯以及 pattern, key, flag_len, key_len 等變數的定義和數值。我將這段反組譯的結果存在 reverse.c 檔案當中。
2. 我們可以看到他的主要邏輯為以下段落：

```
1 if (argc == 2) {  
2     for (i = 0; i < flag_len; ++i) pattern[i] ^= key[i % key_len];  
3     if (!strcmp(argv[1], pattern))  
4         puts("Congratulations! You found the flag!");  
5     else  
6         puts("Haha! wrong >:)!!!!!!");  
7     return 0;  
8 } else {  
9     puts("Usage: ./chal.exe <flag>");  
10    return 1;  
11 }
```

因此，產生 Flag 的邏輯為：將 pattern 陣列中的每個字元與 key 陣列中的字元進行 XOR 運算。

3. 由於我們成功反組譯出 flag_len, pattern 和 key 的值，因此我們可以直接撰寫一個程式來暴力破解出 Flag。我寫了 decrept.cpp 這個程式來完成這個任務。
4. 最後，我得到以下的 Flag：

Flag:

HW12{hW0_8UT_WiTH_r3V3Rse_eN91NE3rinG}